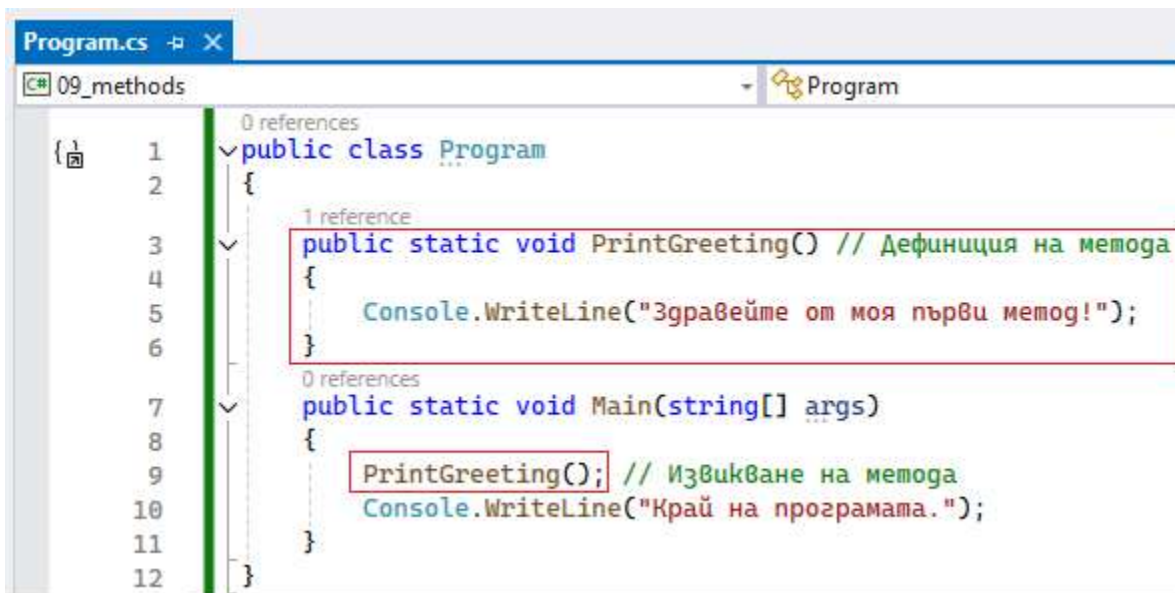


ФУНКЦИИ И МЕТОДИ

Входната точка на програмата е Main()-методът. Той е като съдържанието на книга, от което може да прочетем кое заглавие на коя страница е. Когато харесаме заглавие, отиваме горе, където неговата глава е пакетирана (и кодът е подробен).



Основна концепция в програмирането е да няма повтарящ се код – DRY – don't repeat yourself. Например, когато даден код, който ще ни трябва повече от 1 път, бъде отделен на друго място, ограден в рамките на функция, а когато ни потрябва, само да извикаме нейното име и ѝ вложим необходимите стойности.

Ето основните причини, поради които методите са незаменими:

Повторна употреба на код (Reusability): Вместо да пишете един и същ код многократно, можете да го опаковате в метод и да го извиквате (използвате) навсякъде, където ви е нужен. Това спестява време и намалява грешките.

Организация и четливост: Методите разбиват сложни задачи на по-малки, управляеми части. Това прави кода по-лесен за четене, разбиране и дебъгване.

Поддръжка: Ако откриете грешка или трябва да промените дадена функционалност, променете я само на едно място (в метода), вместо да търсите и редактирате множество повтарящи се блокове код.

Модулност: Методите насърчават модулния дизайн, където всяка част от програмата има ясно дефинирана отговорност.

Всъщност, ние сме използвали много вградени функции досега. Какво означава това? Че вместо ние да пишем, например кода за сравняване на 2 числа а и b с if-else, някой преди нас е преценил, че това ще се използва често, написал е кода, пакетирал го е между блок с отваряща и затваряща

кдрави скоби {} дал му е име Min или Max и на нас остава само да извикаме функцията поименно и да ѝ подадем (поставим) необходимите променливи, които тя да обработи и да ни върне резултата от тях. Така, кодът става много по-четим и логически подреден, защото, вместо да го утежняваме с повтарящи се конструкции, просто извикаме името колкото пъти и когато ни трябва.

Някои от най-често използваните вградени математически функции

// Вградени математически функции, които се използват често

```
int a = 2;
int b = 3;
double c = 4.5;
double d = 5.6;
```

При натискане на точка, се отваря контекстно меню за избор. Ако го изгубим - `backspace` и дописваме отново

```
int e = int.MinValue; // най-малкото възможно число int (при сортиране)
```

```
int f = int.MaxValue; // най-голямото възможно int
```

типът int има вградени функции, които извикваме чрез точка •

```
int min = int.Min(a, b); // по-малкото от 2 числа
```

```
int max = int.Max(a, b);
```

```
int abs = int.Abs(a); // Absolute, абсолютна стойност, модул - без +
```

```
double minD = Math.Min(c, d);
```

Math. е основна математическа функция, която чрез наследяване - точка, извиква подфункции

```
double maxD = Math.Max(c, d);
```

```
double absD = Math.Abs(c);
```

```
double pi = Math.PI; // 3.14 с точност до 16-я знак след
```

```
double round = Math.Round(c); // закръгля със стандартна точност 0.5
```

```
double round2 = Math.Round(c, a); // закръгля до a-брой знаци след запетаята
```

```
double floor = Math.Floor(c); // (пог) закръгля към по-малкото число 3.7 -> 3.0
```

```
double ceiling = Math.Ceiling(c); // (маван) закръгля към по-голямото число 4.2 -> 5
```

```
double squareRoot = Math.Sqrt(a); // корен квадратен
```

```
double powered = Math.Pow(a, b); // степенуване a ^ b
```

`double sinus = Math.Sin(c);` | има още много вградени математически функции, които извикваме

`double cosus = Math.Cos(c);` по име; всяка може да има по няколко възможности overloads

Ако не сме сигурни даден вграден метод какво точно прави, задържаме курсора върху него:

```
int.Min(a, b); // по-малкото от 2 числа
```

```
int Min(int x, int y)
```

Compares two values to compute which is lesser.

$$ID = M$$

cD = M Returns:

$$D = M \times$$

= Mat if it is less than

und =

$$\text{ind2} = y$$
$$10r = \begin{cases} 1 & \text{otherwise} \end{cases}$$

Blank

ling y

lareRo

vered

Thus =

sus = [GitHub Examples and Documentation \(Alt+O\)](#)

Отпечатваме на конзолата по интересен начин – чрез комбинация от `$@`. Знакът `$` ни е познат (за смес между текст и променливи), а `@` verbatim (от verb) е дословно печатане – както сме го изписали с празни редове, интервали, табулации, така ще го разпечата. По този начин се избягва утежняването на кода с повтарящи се конструкции и команди като `Console.WriteLine` за всеки нов ред или `\n` – new line, `\t` – табулация и т.н. Без знак `@`, всички резултати ще се изпечатат на конзолата един след друг, слято.

```
Console.WriteLine($@ -
a = {a}
b = {b}
c = {c} интерполация и
d = {d} дословно

e = int.MinValue = {e}
f = int.MaxValue = {f}

int.Min(a,b) = {min}
int.Max(a,b) = {max}
int.Abs(a) = {abs}
Math.Min(c,d) = {minD}
Math.Max(c,d) = {maxD}
Math.Abs(c) = {absD}
Math.PI = {Pi}
Math.Round(c) = {round}
Math.Round(c, a) = {round2}
Math.Floor(c) = {floor}
Math.Ceiling(c) = {ceiling}
Math.Sqrt(a) = {squareRoot}
Math.Pow(a, b) = {powered}
Math.Sin(c) = {sinus}
Math.Cos(c) = {cosus}
");
```

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace ConsoleApp1
{
    class Program
    {
        static void Main()
        {
            a = 2
            b = 3
            c = 4.5
            d = 5.6

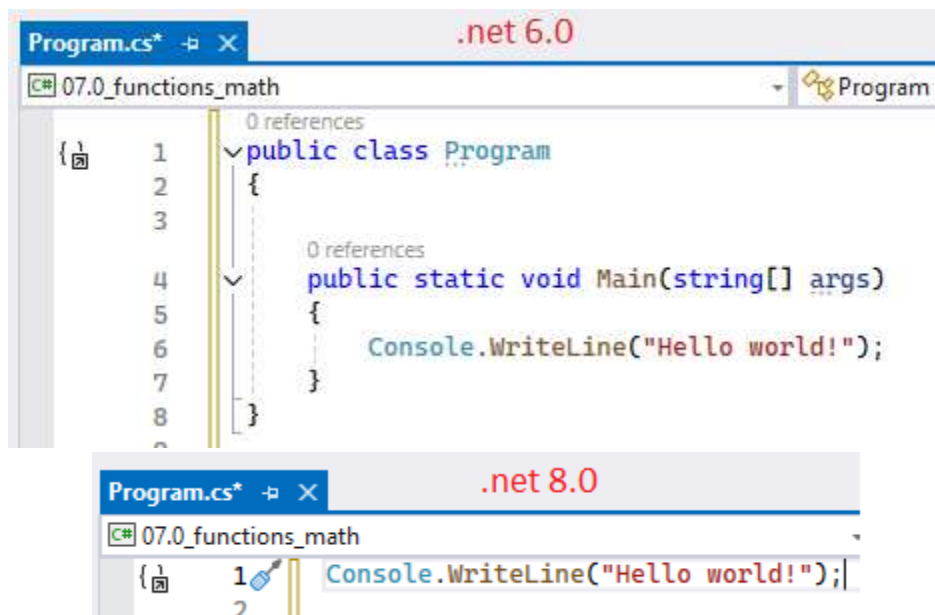
            e = int.MinValue = -2147483648
            f = int.MaxValue = 2147483647

            int.Min(a,b) = 2
            int.Max(a,b) = 3
            int.Abs(a) = 2
            Math.Min(c,d) = 4.5
            Math.Max(c,d) = 5.6
            Math.Abs(c) = 4.5
            Math.PI = 3.141592653589793
            Math.Round(c) = 4
            Math.Round(c, a) = 4.5
            Math.Floor(c) = 4
            Math.Ceiling(c) = 5
            Math.Sqrt(a) = 1.4142135623730951
            Math.Pow(a, b) = 8
            Math.Sin(c) = -0.977530117665097
            Math.Cos(c) = -0.2107957994307797
        }
    }
}
```

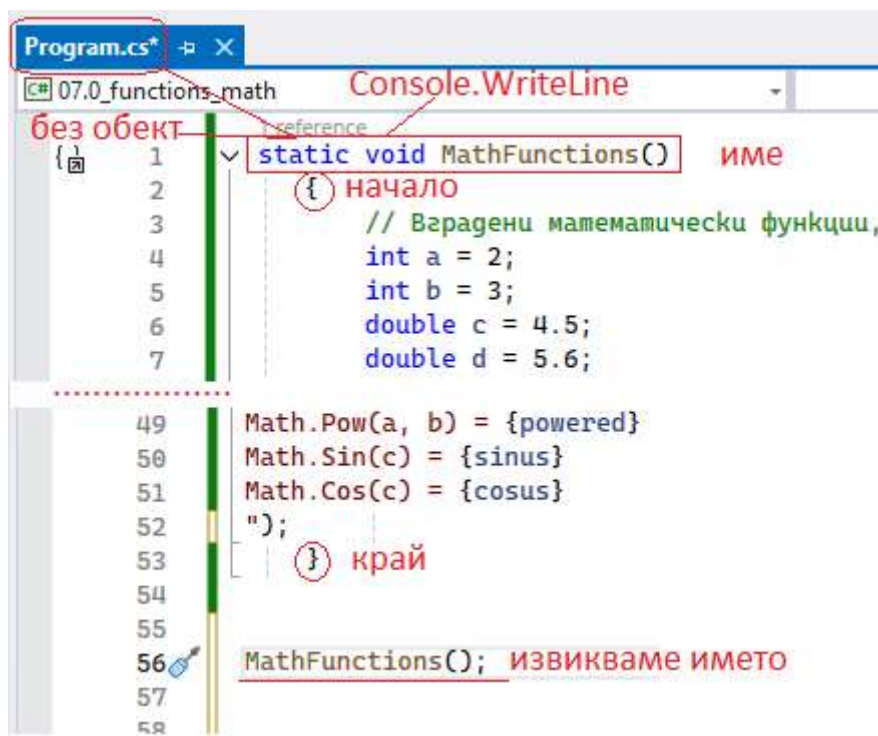
! Теоретичната разлика между функция и метод е, че методът връща резултат със стойност (`int`, `double`) докато функцията – не (`void`) – тя само извършва действие. Това е чисто формално и нестрого обяснение, т.к. в различните езици за програмиране, двете понятия се използват по различен начин и често са синоними спрямо тази подробност. В `C#` по-препоръчително е да използваме „метод“ за всичко – така ще сме прави в 99% от случаите, докато в `JavaScript`, например, почти винаги се използва „функция“ за абсолютно същите конструкции.

Сега може да разгледаме как да превърнем този код във функция:

- 1) заграждаме кода между `{ ...код ... }`
- 2) намираме се в `Program.cs` – файла, т.е. клас `Program` и в неговия метод `Main()`, който се вижда явно в `.NET 6.0` и по-старите версии, но от `.NET 8.0` и по-новите, е скрит.



Т.е. ако работим в новата .Net 8.0 може да се наложи ние да си допишем class Program + Main().



Може да скрием кода със стрелката вляво – във версия .net 8.0 е достатъчно да напишем само static – т.е. намираме се в Program.cs, void – не връщаме никаква стойност, а само отпечатваме с конзолата, т.е. запомнете, че **void се използва при Console.WriteLine**.


```
Program.cs* [X]
C# 07.0_functions_math
1 static void MathFunctions() ...
54 { скрити 53 реда код }
55
56 MathFunctions();
--
```

Т.е. в 8.0 е достатъчно да напишем

```
Program.cs* [X]
C# 07.0_functions_math
1 static void MathFunctions() ...
54
55
56 MathFunctions();
57
58
```

```
Program.cs* [X]
C# 07.0_functions_math
1 static void MathFunctions() ... първо код
54
55
56 MathFunctions(); после извикване
57
```

В по-старите версии се вижда името на класа Program (същото име като файла Program.cs) и входната точка на програмата Main()-метода:

```
Program.cs [X] .net 6.0
C# 07.0_functions_math
1 public class Program
2 {
3     public static void Main(string[] args)
4     {
5         static void MathFunctions() ...
58
59         MathFunctions();
60     }
61 }
```

По принцип, в public static void Main() само извикваме имената на методите, които искаме – така е по-подредено и четимо, докато кодът, който върши работата, си има свои public static-имена() {} по същия начин като Main и са отделени в class Program:

```
Program.cs*  X
C# 07.0_functions_math  Progra
0 references
1 public class Program
2 {
3     > static void MathFunctions()...  КОДЪТ
4                                     ВЪН
56 public static void Main(string[] args)
57 {
58     MathFunctions();
59 }
60 }
```

Може също да го декорираме с public:

```
Program.cs*  X
C# 07.0_functions_math  Progra
0 references
1 public class Program
2 {
3     > public static void MathFunctions()...
4
56 public static void Main(string[] args)
57 {
58     MathFunctions();
59 }
60 }
```

Т.е. като цяло, ние имаме готов клас Program, който има 1 метод в него Main(), приет за начало. Подобно на него правим други методи, които се подреждат над Main(), а вътре е него само ги викаме по име – може по няколко пъти, може и с различни стойности.

```

1  public class Program
2  {
3      /* public static void ИмеНаФункция1()
4      {
5          Console.WriteLine("Здравейте, XI клас!");
6      }
7
8      public static int ИмеНаФункция2()
9      {
10         връз ког, който връща
11         return цяло число;
12     }
13
14     public static double ИмеНаФункция3(int a, double b)
15     {
16         някакви изчисления с int и double;
17         return връща резултат double;
18     }*/
19
20     public static void Main(string[] args)
21     {
22         ИмеНаФункция1();
23         ИмеНаФункция2();
24         ИмеНаФункция3(4, 5.8);
25     }
26 }

```

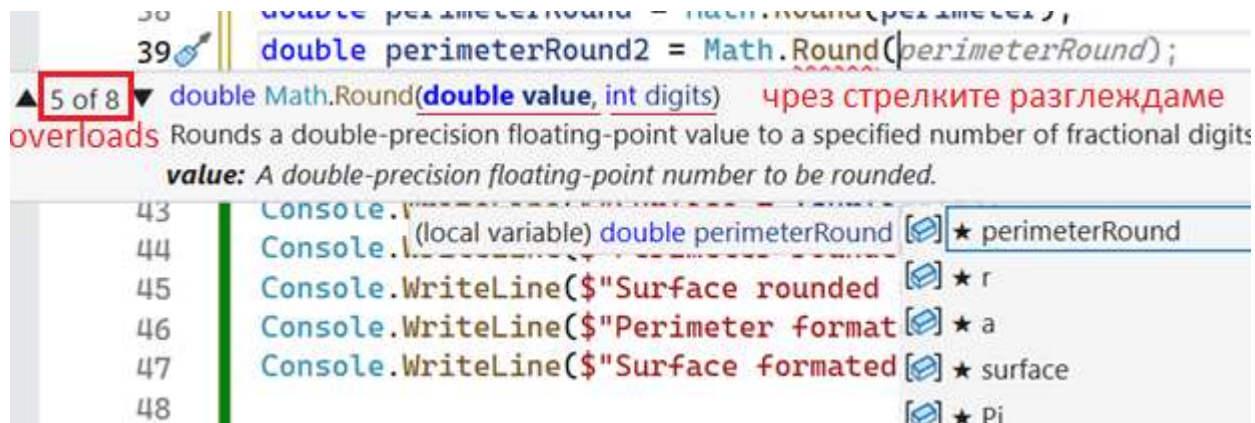
над Main() отделяме кодовете

в Main() само извикваме имената им и подаваме стойности

Задача: Използвайки вградените математически функции, изчислете обиколката (периметър) и лицето (surface) на кръг до 3-я знак след десетичната запетая. Ако се колебае коя е правилната формула, си спомнете, че дължината/обиколка е в първо измерение – $2 * \pi * r \rightarrow \text{см}$, т.е. радиусът се среща само веднъж, т.е. имаме само 1 вектор за дължина и отговорът е в първа степен см^1 – това може да бъде само и единствено линия. Лицето е в 2 измерения (нещо по нещо) -> $\pi * r^2 \rightarrow \text{см}^2$, защото имаме 2 пространствени вектора за дължина и ширина, т.е. $r * r$ – образува се вече фигура (кръг, правоъгълник и т.н.), която може да бъде изобразена на лист хартия. При обем volume V векторите за пространство са 3: ширина, дължина и височина, съответно отговорът е в см^3 .



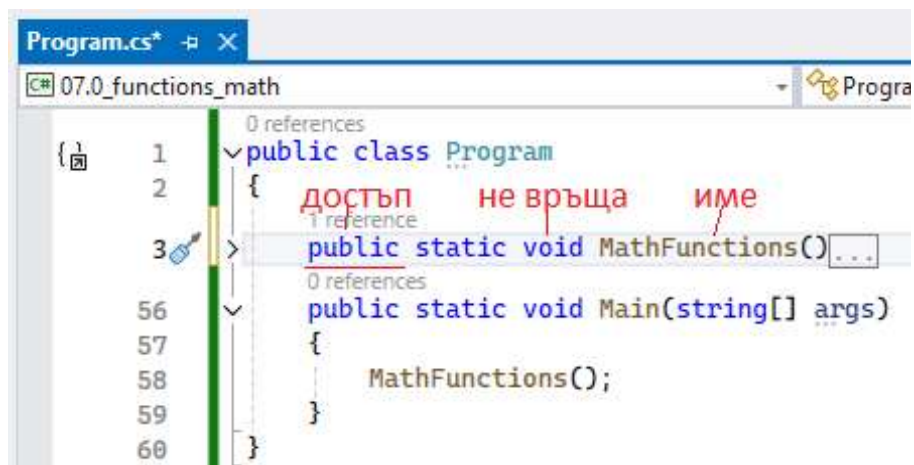
Използвайки авто-дописването IntelliSense с подсказки за дописване може да разглеждаме вариантите на вградените функции. Така, например, Math.Round има 8 варианта за използване overloads, които може да разглеждаме чрез стрелките и четем как се прилагат. Има варианти само с подаване на 1 десетична променлива, но има и варианти, в които освен нея, подаваме и цяло число с точност до какъв брой знаци след запетаята да бъде закръглянето.



Какво означава това – че кодът, който върши работата, е отделен (скрит) на друго място, а ние просто го извикваме чрез името му и поставяме променливите, които да обработи. Тези променливи могат да се подадат директно (с цифри) или неявно.

тип ИмеНаФункция (параметър1, параметър2, ... параметърN)

```
{
    // ...
    // тук с код
    return израз;
}
```

където:

тип е стандартен тип, включително void (празен тип). Ако типът е пропуснат, се подразбира int.

Модификатори за достъп (Access Modifiers): Определят откъде може да бъде извикан методът.

public: Методът е достъпен от всяко място. Той е най-често срещаният и подходящ за начинаещи.

private: Методът е достъпен само в рамките на класа, в който е дефиниран.

protected: Достъпен в рамките на класа и в наследяващи класове.

internal: Достъпен в рамките на текущия проект (assembly).

Тип на връщаната стойност (Return Type):

Това е типът данни, който методът ще върне като резултат след изпълнението си. Може да бъде int, string, double, bool или някой друг тип данни (включително сложни обекти).

Ако методът не връща никаква стойност, се използва ключовата дума **void**. Тя се използва, когато методът само принтира съобщение на конзолата Console.WriteLine().

ИмеНаМетода (MethodName):

Трябва да е описателно и да отразява какво прави методът (напр. CalculateSum, PrintMessage, GetUserAge). В C# е прието имената на методите да започват с главна буква (**PascalCase**).

(Параметри) (Parameters):

Това са входните данни, които методът очаква, за да изпълни задачата си. Параметрите са списък от тип данни и име на променлива, разделени със запетаи. Ако методът не изисква входни данни, скобите се оставят празни ().

Примери:

```

✓ static int Sum(int a, int b) // връща int
{
    return a + b;
}
    (очаква да се подадат a и b)
    2 references
    (трябва да получи a)
✓ static int Square(int a) // връща int
{
    return a * a;
}
    1 reference
✓ static void Print(string username) // не връща, печата
{
    Console.WriteLine($"Hello, {username}!");
}

```

Вече може да извикваме тези функции по име и подадени стойности:

```

Console.WriteLine("Мемог и функции");
string user = Console.ReadLine();
Print(user);

```

```

Sum(10, 20); // извиква метод и му задава стойности
Square(4);

```

```

int x = Sum(20, 30); // присвоява метод на променлива
int y = Square(5);

```

```

Console.WriteLine(x); // печата променлива
Console.WriteLine(Sum(30, 40)); // печата метод

```

```

Console.WriteLine(y);

```

В контекста на C# - НЯМА съществена разлика между метод и функция. Двата термина се използват взаимозаменяемо, но има леко семантично различие. В документацията на Microsoft и официалните източници се използва предимно (99%) "метод", която е по-официална.

Термин	Дефиниция	Пример в С#
Функция	Блок код, който приема параметри и връща резултат	int Multiply(int x, int y)
Метод	Функция, която е член на клас/обект	calculator.Multiply(5, 3)
Procedure	Блок код, който извършва действие (не връща стойност)	void DisplayMessage()

Аналогично може да напишем код за намиране на обиколка и лице на кръг, както и обем на сфера по зададен радиус:

```

3 references
static void Circle(double r)
{
    double perimeter = 2 * Math.PI * r;
    double surface = Math.PI * Math.Pow(r,2); //3.14 * r * r
    //double volume = (double)4/3 * Math.PI * Math.Pow(r,3); - for sphere

    double perimeterRound = Math.Round(perimeter);
    double perimeterRound3Digits = Math.Round(perimeter, 3);
    double surfaceRound = Math.Round(surface);

    Console.WriteLine($" === Results ===
    Perimeter = {perimeter}
    Perimeter rounded = {perimeterRound}
    Perimeter ROUNDED 3 digits = {perimeterRound3Digits}
    Perimeter FORMATED 3 digits = {perimeter:F3}

    Surface = {surface}
    ");
}

```

```

07.0_functions_math\Program.cs (3)
ред 104 : Circle(a);
        106 : Circle(21);
        110 : Circle(test);
Collapse All
*/
3 references 3 референции/извиквания
static void Circle(double r)
{

```

Съответно можем да извикваме променливите по различни начини – с директно подаване на стойности или индиректно с променливи:

```

Console.Write("Enter radius r = ");
double r = double.Parse(Console.ReadLine());
int a = 7;

Circle(a); 1      извикване на метода
Circle(21); 2      по различни начини

double test = 1.5;

Circle(test); 3      пъти

Console.WriteLine("END");

```

```

Microsoft Visual Studio Debug Console
Enter radius r = 4
=== Results === 1
Perimeter = 43.982297150257104
Perimeter rounded = 44
Perimeter ROUNDED 3 digits = 43.982
Perimeter FORMATED 3 digits = 43.982

Surface = 153.93804002589985

=== Results === 2
Perimeter = 131.94689145077132
Perimeter rounded = 132
Perimeter ROUNDED 3 digits = 131.947
Perimeter FORMATED 3 digits = 131.947

Surface = 1385.4423602330987

=== Results === 3
Perimeter = 9.42477796076938
Perimeter rounded = 9
Perimeter ROUNDED 3 digits = 9.425
Perimeter FORMATED 3 digits = 9.425

Surface = 7.0685834705770345

END

D:\Exercises\11 class\09_exam_preparation\b
Press any key to close this window . . .

```

Ако при написването на кода, все още името му никъде не е извикано, изписва, че има 0 референции към него. Ако има референции, чрез линк върху надписа ще ги посочи

```

0 references не е извикан все още
static void MathFunctions()
{
    // Взрадени математически фу

```


Накрая може да си подредим извикванията на двата метода:

```
Console.WriteLine("===== Math Functions =====");

MathFunctions();

Console.WriteLine("===== END =====");

▼ Console.Write(@"Do you want to continue?
  Y/N: ");
char answer = char.Parse(Console.ReadLine());
if (answer == 'N' || answer == 'n') return; // ранен край
▼ else
{
    Console.Write("Enter radius r = ");
    double r = double.Parse(Console.ReadLine());
    int a = 7;

    Circle(a);

    Circle(21);

    double test = 1.5;

    Circle(test);
}
Console.WriteLine("===== END ====="); // само слег Yes
```